

DESIGN-OS: A SPECIFICATION-DRIVEN FRAMEWORK FOR ENGINEERING SYSTEM DESIGN WITH A CONTROL-SYSTEMS DESIGN CASE

H. Sinan Bank*

Department of Systems Engineering
Colorado State University
Fort Collins, CO 80523

Daniel R. Herber

Department of Systems Engineering
Colorado State University
Fort Collins, CO 80523

Thomas H. Bradley

Department of Systems Engineering
Colorado State University
Fort Collins, CO 80523

ABSTRACT

Engineering system design—whether mechatronic, control, or embedded—often proceeds in an ad hoc manner, with requirements left implicit and traceability from intent to parameters largely absent. Existing specification-driven and systematic design methods mostly target software, and AI-assisted tools tend to enter the workflow at solution generation rather than at problem framing. Human–AI collaboration in the design of physical systems remains underexplored. This paper presents Design-OS, a lightweight, specification-driven workflow for engineering system design organized in five stages: concept definition, literature survey, conceptual design, requirements definition, and design definition. Specifications serve as the shared contract between human designers and AI agents; each stage produces structured artifacts that maintain traceability and support agent-augmented execution. We position Design-OS relative to requirements-driven design, systematic design frameworks, and AI-assisted design pipelines, and demonstrate it on a control systems design case using two rotary inverted pendulum platforms—an open-source SimpleFOC reaction wheel and a commercial Quanser Furuta pendulum—showing how the same specification-driven workflow accommodates fundamentally different implementations. A blank template and the full design-case artifacts are shared in a public repository to support reproducibility and reuse. The workflow makes the design process visible and auditable, and extends specification-driven orchestration of AI from software to physical engineering system design.

Keywords: Design Theory and Methodology; Design Process; Systems Engineering; AI/KBS; Control; Mechatronics and Electro-Mechanical Systems; Design Automation

1 INTRODUCTION

When design is ad hoc or implementation-first, requirements tend to remain implicit, and traceability from intent to parameters is difficult to achieve. AI-assisted tools compound this tendency: evidence from the design literature indicates they are predominantly applied in later solution-generation stages rather than in problem framing and specification [1]. Yet the same AI capabilities, when guided by structured specifications, can support better and earlier alignment on goals—recent work on conceptual systems engineering illustrates the value of human-driven baselines alongside agentic frameworks [2]. What is missing is an orchestration layer for the design process itself: a lightweight, model-agnostic structure that sequences design activities, enforces specification-before-implementation discipline, and provides a common interface through which both human designers and AI agents operate. This paper presents *Design-OS*, a lightweight, specification-driven, phased workflow that addresses this gap by making specifications the binding contract between conceptual intent and implementation and by front-loading literature and conceptual design before committing to a specification. *Design-OS* adopts this collaborative, specification-first stance while remaining compatible with agentic tools where they add value, and structures the process so that traceability from intent to parameters can be agnostically documented and converted for other schemas or frameworks.

Design-OS proceeds in five stages: concept definition → literature survey → conceptual design → requirements definition → design definition, with selected stage terminology aligned with INCOSE SE Handbook processes (notably Design Definition and Requirements Definition) [3]. It is intended for engineering design, research prototypes, education, and AI-augmented design workflows. By using a control system design project as a design case, we demonstrate how the workflow supports the kind of structured design process and traceability that engineering design

*Corresponding author: sinan.bank@colostate.edu

methodology calls for [4], in a setting where prior work on this benchmark has focused largely on control algorithms and performance analysis [5] rather than on embedding the design process in a formal, traceable workflow. This paper addresses three research questions:

RQ1. Why is a lightweight, specification-driven workflow needed for physical engineering system design? (Sec. 1–2)

RQ2. How does *Design-OS* relate to existing specification-driven, systematic-design, and AI-assisted design frameworks? (Sec. 2–3)

RQ3. How does the workflow support traceability from intent to parameters across domains? (Sec. 4–5)

Aligning with these questions, the contributions of this paper are: (1) a concise presentation of the *Design-OS* methodology and its positioning relative to specification-driven and requirements-driven design (including industry practice such as GitHub Spec Kit [6] and Microsoft specification-driven development [7]), systematic design frameworks (Pahl & Beitz [4], VDI [8], Stage-Gate [9]), and AI-assisted design. (2) An end-to-end design case showing how the workflow was run and how artifacts trace from context and literature through to requirements and implementation plan. (3) A discussion of implications for design education and for specification-driven orchestration of AI in physical engineering system design.

The remainder of the paper is organized as follows: Section 2 reviews related work. Section 3 describes the *Design-OS* methodology. Section 4 presents the inverted pendulum design case. Section 5 discusses limitations and future work, and Section 6 concludes.

2 RELATED WORK

Specification-driven and requirements-driven design. The principle that formal specifications must precede implementation is well established in requirements engineering and systems engineering [10, 11]. Traceability from requirements to design and verification is central; inadequacy of pre-requirements specification traceability is a major source of failure [12]. Reference models for traceability have since formalized the relationships among requirements, design decisions, and verification artifacts [13]. Standards such as IEEE 830 [14] and its successor ISO/IEC/IEEE 29148 [15] define quality criteria and templates for software and system requirements specifications. Structured syntax approaches such as EARS provide five requirement templates (precondition–trigger–shall–response) to reduce ambiguity and vagueness in natural-language requirements [16]. Industry practice has recently emphasized specification-driven development (SDD) in the age of AI coding assistants, treating specifications as the source of truth from which code is generated or verified rather than the reverse [17]: **Microsoft** promotes specifications as “version control for your thinking,” with technical decisions made explicit

and reviewable before code becomes the de facto specification [7]. The open-source **GitHub Spec Kit** provides a toolkit for outlining requirements, motivations, and technical aspects before handing work to AI agents (e.g., GitHub Copilot), including a “Constitution” for engineering guidelines, clear specifications, technical plans, and implementation tasks [6]. *Design-OS* aligns with this philosophy and extends it from software to *physical* engineering system design, with stages—literature review, conceptual design of mechatronic or control systems, performance specifications—that are domain-specific.

Systematic design methodology. *Design-OS*’s phased workflow draws on systematic design frameworks. Pahl and Beitz’s four-phase model (task clarification, conceptual design, embodiment design, detail design) is reflected in *Design-OS*’s early phases [4]. Gero’s Function-Behavior-Structure (FBS) knowledge representation schema provides a complementary theoretical lens, formalizing how design transforms functions into structural descriptions through expected and actual behaviors [18]. VDI 2206 (V-model for mechatronics and cyber-physical systems) prescribes phased deliverables and continuous requirements management [8]. *Design-OS* aims to compress these into a workflow optimized for specification-driven engineering. Stage-Gate [9] shares the idea of discrete phases with explicit deliverables and front-loading. *Design-OS* focuses on the technical design workflow rather than the full product lifecycle. *Design-OS* does not replace these frameworks but offers a lightweight, AI-friendly instantiation that can be used on its own or alongside MBSE and VDI, where a lighter-weight, specification-driven workflow is desired.

AI-assisted design and the specification-driven gap. Large language models and multi-agent frameworks are reshaping design workflows. Multi-agent systems such as MetaGPT and ChatDev use structured intermediate outputs (requirement documents, system designs, interface specs) to orchestrate agents and reduce cascading errors [19, 20]—directly analogous to *Design-OS*’s phased artifacts. A notable gap is that specification-driven orchestration of AI agents for *physical* engineering system design has received little attention. Existing multi-agent pipelines target software development [21]. Recent work has shown that LLMs can generate conceptual design solutions with higher average feasibility and usefulness than crowdsourced alternatives, though crowdsourced solutions exhibit more novelty [22], and that generative pre-trained transformers can produce diverse design concepts from textual prompts [23]—yet these capabilities remain disconnected from structured specification workflows. Studies of human–AI collaboration in design indicate that most AI design tools support later solution-generating stages, with few supporting early-stage problem discovery [1]. *Design-OS* addresses this gap by providing a workflow that spans from concept definition and literature survey through to design definition, extending the

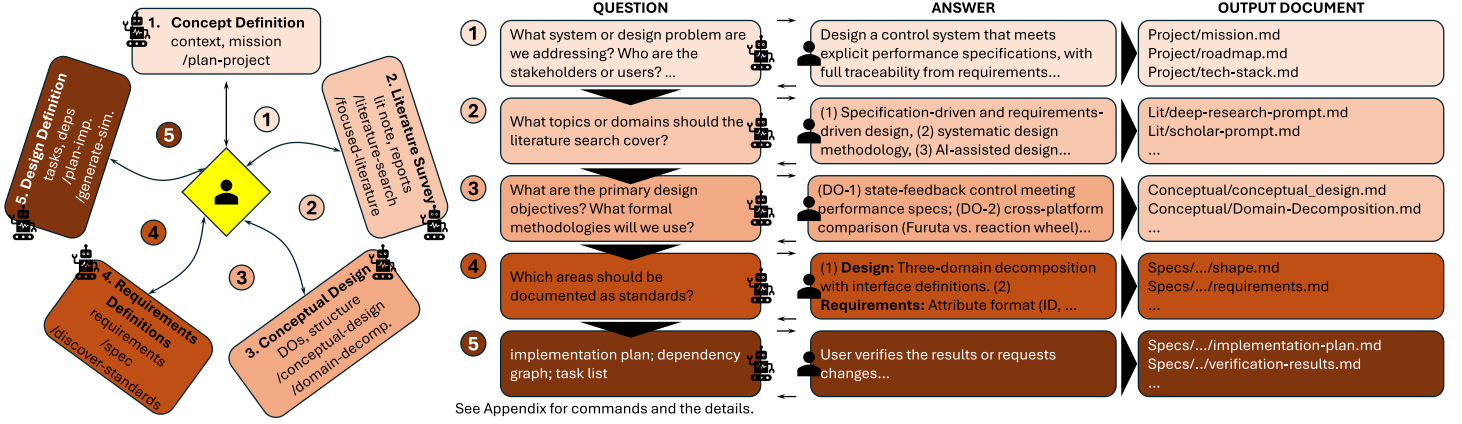


FIGURE 1: *Design-OS* workflow (abstract): five stages arranged around the designer (central diamond), each executed by an AI agent (robot icon). User interactions (numbered) between stages enable validation and iteration. Each box shows the stage name, key artifacts, and commands.

specification-driven multi-agent paradigm to physical engineering design.

Control education and the design case. Control systems engineering and education have long relied on canonical benchmark platforms—the inverted pendulum, ball-and-beam, ball-and-plate, and servo systems—to teach dynamics, underactuation, and feedback design [24, 25]. Commercial platforms are widely deployed, and open-source alternatives are also available. Control education methodology emphasizes interactivity and structured use of technology [26]. Yet these systems are rarely situated within a *formal, traceable* design process: the mapping from performance specifications to controller parameters is typically an in-paper exercise rather than a documented workflow. *Design-OS* targets this gap by providing a specification-driven workflow applicable to any of these platforms. We demonstrate it on two inverted pendulum platforms (Furuta and reaction wheel) and share a blank template that can be adapted to other projects.

3 METHODOLOGY: DESIGN-OS

Design-OS is a specification-driven, phased workflow for engineering system design. Each stage produces explicit artifacts that inform the next, supporting a path to traceability from initial motivations to the final implementation plan (realizing such traceability in practice remains challenging and tool-dependent). The workflow is designed to be lightweight enough for research projects, capstone design, and rapid iteration, while retaining the rigor of formal requirements management.

Stage 1: Concept Definition. The designer captures context, mission, and scope in a structured project folder (e.g., `mission.md`, `roadmap.md`, `tech-stack.md`). This stage establishes the problem statement, stakeholders, success criteria,

and constraints. It parallels the Business or Mission Analysis process in INCOSE [3], task clarification in Pahl & Beitz [4], and problem identification in DSRM [27].

Stage 2: Literature Survey. An initial and focused literature search is conducted, aligned with the project goals. Artifacts include a preliminary literature note, deep-research prompts, scholar search queries, and literature reports. The designer formulates structured research prompts describing the topics and gaps to investigate. AI research assistants (e.g., ChatGPT Deep Research, Claude, Gemini) then execute these prompts and *return* candidate references, summaries, and gap analyses. The designer curates the results—verifying that cited works exist, removing hallucinated references, and merging findings into the preliminary literature note. The key outcome is a landscape of prior work, identified gaps, and references that feed conceptual design. This stage enforces that design decisions are grounded in existing knowledge before any specification is written.

Stage 3: Conceptual Design. Design objectives (DOs), product or publication structure, and a positioning narrative are developed. The designer determines what the artifact is, how it relates to prior work, and what gap it fills. In a control design context, this stage identifies candidate modeling approaches, control strategies, and system architectures. The conceptual design is fixed before committing to the requirements definition, reducing downstream rework.

Stage 4: Requirements Definition. Stage 4 proceeds in two substeps: *shape* (define section-level requirements and structure) and *elaborate* (fill in detailed requirements with IDs, “shall” statements, source, priority, and verification method). The specification also documents functions, architecture, interfaces, and

validation criteria. The specification is the *binding contract* between intent and implementation: it serves as the single source of truth that governs all downstream work. Standards discovered during the process (e.g., requirements format, system structure templates) are documented alongside the specification.

Stage 5: Design Definition. A task breakdown, dependency ordering, and execution plan are derived from the specification. Each task maps to a specific deliverable and to the requirement(s) it satisfies. In a control design project, the plan might include: derive a mathematical model, design controller gains from requirements, build a simulation, run verification, and document traceability.

Traceability across stages. Traceability is maintained by linking requirement IDs to design parameters and to verification steps (e.g., REQ-001 $\leftrightarrow$ settling time ≤ 1.0 s $\leftrightarrow$ state-feedback gain $K \leftrightarrow$ simulation step response). The process is iterative within stages, but stage order is respected: no design definition is written before an agreed requirements definition, and no requirements definition before a conceptual design. Figure 1 summarizes the stages and their main artifacts.

4 DESIGN CASE: INVERTED PENDULUM

We applied the *Design-OS* workflow to an inverted pendulum control design project—a canonical benchmark in control education and research [5, 28].

The AI-assisted literature search (Stage 2) mapped a landscape of commercial and open-source platforms for control education, including Quanser (rotary and linear pendulums) [29], Feedback Instruments (Digital Pendulum) [30], and several open-hardware projects (SimpleFOC reaction wheel pendulum, 3D-printable Furuta pendulums, cart-pole kits). From this landscape, two platforms were selected at the feasibility gate (Stage 3): (1) the **Quanser SRV02** Furuta-type rotary inverted pendulum, chosen for its extensive academic track record—deployed in hundreds of university labs worldwide with ABET-aligned courseware and manufacturer-documented parameters—and (2) the **SimpleFOC-based reaction wheel inverted pendulum** [31, 32], chosen for its open-source hardware and software, comprehensive build documentation, active community, and validated control examples [33]. This selection demonstrates that the same specification-driven workflow and requirements can be satisfied by fundamentally different implementations—one commercial, one open-source. Full artifacts for this design case, along with a blank *Design-OS* template for other potential projects, are available in a public GitHub repository [34].

4.1 Stage 1–2: Concept Definition and Literature Survey

Context. The mission was defined as follows: design a control system for a rotary inverted pendulum that meets explicit performance specifications, with full traceability from requirements to controller parameters across the mechanical, electrical, and software domains. The roadmap established five phases (concept definition, literature survey, conceptual design, requirements definition, design definition), and the tech stack was defined as MATLAB/Simulink for modeling and control, with Python as an alternative.

Literature. A literature search targeted four areas: (i) specification-driven and requirements-driven design, (ii) systematic design methodology, (iii) AI-assisted design, and (iv) control education and the inverted pendulum benchmark. Structured deep-research prompts were formulated and executed by multiple AI research assistants (Claude, Gemini). Each assistant independently searched for and returned candidate references, which were then verified by the designer and merged into a preliminary literature note. The designer specified the research topics; the AI assistants independently identified the specific references within those topics. The following key papers were *surfaced by the AI-assisted search* and, after verification, informed the conceptual design: Furuta et al. [28] for the original Furuta pendulum, Cazzolato and Prime [35] for the definitive dynamics derivation, Hamza et al. [5] for the benchmark survey, Quanser’s rotary pendulum workbook for hardware parameters and lab specifications, Skuric et al. [32] for the SimpleFOC library and its example implementation underlying the open-source reaction wheel platform [31], and Wright [33] for a validated reaction wheel pendulum platform designed for controls education using SimpleFOC.

4.2 Stage 3: Conceptual Design and Domain Decomposition

Three design objectives were defined:

(DO-1) What state-feedback control design satisfies the performance specifications for each platform?

(DO-2) How do fundamentally different platforms (SimpleFOC reaction wheel vs. Quanser Furuta) satisfy the same set of requirements?

(DO-3) How does cross-domain traceability flow from performance specifications through software, electrical, and mechanical domains to controller parameters?

Two system architectures were identified (Figure 2): (a) a Furuta-type rotary inverted pendulum with state vector $x = [\theta, \alpha, \dot{\theta}, \dot{\alpha}]^T$ (4 states, 2-DOF), and (b) a reaction wheel inverted pendulum with state vector $x = [\theta, \dot{\theta}, \dot{\phi}]^T$ (3 states, 2-DOF), where θ is the pendulum angle and $\dot{\phi}$ is the reaction wheel angular velocity [33].

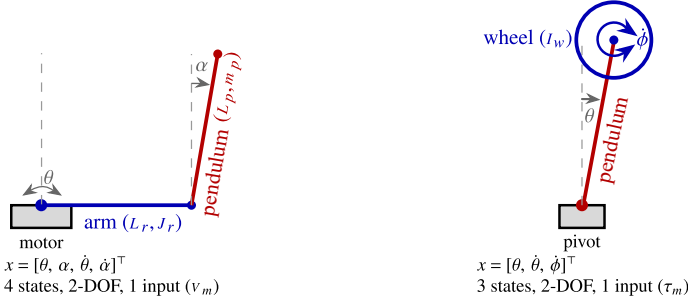


FIGURE 2: Two inverted pendulum architectures: Furuta (left) and reaction wheel (right).

Literature-driven domain discovery. The domain decomposition was driven by the literature *surfaced* in Stage 2—the designer specified the topics, and the AI assistants returned the specific references that established each domain. Furuta et al. [28] and Cazzolato and Prime [35] describe the two-link arm–pendulum system with Euler–Lagrange formulation, establishing the *mechanical* domain. The Quanser SRV02 documentation describes the DC servo motor, gearbox, encoders, and power amplifier—establishing the *electrical* domain and the torque equation in Eq. (1) that couples it to mechanical torque. The control design literature (pole placement, state feedback) and simulation environment (MATLAB/Simulink) establish the *software* domain. Where literature was ambiguous (e.g., whether the DAQ interface board belongs to electrical or software), the designer was consulted at a user checkpoint. The three domains identified are:

Mechanical: rotary arm (length L_r , inertia J_r , damping B_r) and pendulum link (length L_p , mass m_p , inertia J_p , damping B_p). Two-link dynamics derived via the Euler–Lagrange equations.

Electrical: DC servo motor (R_m , k_m , k_t), gearbox (K_g , η_g , η_m), power amplifier, and rotary encoders for θ and α . The torque equation:

$$\tau = \frac{\eta_g K_g \eta_m k_t (V_m - K_g k_m \dot{\theta})}{R_m} \quad (1)$$

couples the electrical domain (voltage V_m) to the mechanical domain (torque τ).

Software: state-feedback control law $u = K(x_d - x)$, state estimation from encoders via differentiation with high-pass filtering, controller enable/disable logic, simulation environment.

The state-space matrices A and B encode the coupling: rows 3–4 of A and B incorporate the motor torque equation in Eq. (1), making the electrical parameters (R_m , k_m , k_t , K_g) part of the plant model. The same three-domain decomposition applies to the reaction wheel pendulum, with different specifics: the mechanical domain is a single-link pendulum with a reaction wheel (inertia I_w) instead of a rotary arm. The electrical domain uses a BLDC motor with field-oriented control (torque constant K_t ,

phase resistance R) instead of a geared DC servo. The software domain uses Arduino/C++ with the SimpleFOC library instead of MATLAB/Simulink.

Feasibility gate. Before proceeding to Requirements Definition (Stage 4), a validation gate was run. The cost comparison (Table 4) was produced at this gate: the SimpleFOC-based reaction wheel pendulum [31] (\$100–200, open-source Arduino/C++ stack) was compared against the Quanser SRV02 platform (\$5k+, proprietary MATLAB/QUARC licensing). The SimpleFOC platform offers accessibility while eliminating licensing constraints. The Quanser platform provides manufacturer-documented parameters and an established lab infrastructure. Wright [33] demonstrated that the SimpleFOC-class platform can be fully characterized through system identification (pendulum inertia, damping, motor torque constant, phase resistance), providing the parameter certainty needed for model-based control design. Software licensing was assessed: Python and Arduino/C++ are freely available; MATLAB/Simulink is proprietary. The designer confirmed at a user checkpoint that both platforms would be carried forward: the SimpleFOC reaction wheel pendulum as the primary open-source platform, and the Quanser as a well-documented commercial comparison. A Python simulation was specified for reproducibility.

4.3 Stage 4: Requirements Definition

The Requirements Definition stage produced requirements organized by domain (Table 1). Performance requirements (REQ-01–04) are system-level. Domain-specific requirements (mechanical, electrical, software) ensure that the full mechatronic design is captured and traceable.

Modeling (mechanical + electrical). The nonlinear equations of motion are derived via the Euler–Lagrange method. Letting $q = [\theta, \alpha]^T$, the dynamics take the matrix form:

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau, \quad (2)$$

where D is the inertial matrix, C the damping/Coriolis matrix, g the gravitational vector, and τ the torque vector. After linearization about the upright equilibrium and incorporation of the motor torque in Eq. (1) (REQ-05, REQ-07), the linear state-space model is:

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (3)$$

where $u = V_m$. Using Quanser SRV02 parameters:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 80.3 & -45.8 & -0.93 \\ 0 & 122 & -44.1 & -1.40 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 83.4 \\ 80.3 \end{bmatrix}. \quad (4)$$

The entries in rows 3–4 reflect the coupling of mechanical parameters (m_p , L_p , J_r , J_p) with electrical parameters (R_m , k_m , k_t , K_g). The open-loop poles $\{7.12, 0, -5.95, -48.37\}$ confirm instability.

TABLE 1: Requirements by domain for the rotary inverted pendulum.

ID	Domain	Requirement (shall)	Verif.
REQ-01	Perf.	Damping ratio $\zeta = 0.7$	Sim.
REQ-02	Perf.	Natural freq. $\omega_n = 4$ rad/s	Sim.
REQ-03	Perf.	$ \alpha < 15$ deg	Sim.
REQ-04	Perf.	$ V_m < 10$ V	Sim.
REQ-05	Mech.	State-space from first-principles EOM + linearization	Review
REQ-06	Mech.	Parameters documented with source	Review
REQ-07	Elec.	Actuator dynamics (R_m, k_m, k_t, K_g) in model	Review
REQ-08	Elec.	Encoders measure θ, α	Review
REQ-09	SW	Control at sufficient sampling rate	Review
REQ-10	SW	Velocity estimation with filtering	Review
REQ-11	SW	Gains from requirements (pole placement)	Review
REQ-12	All	Traceability table: REQ $\rightarrow$ param $\rightarrow$ verif.	Review

Reaction wheel pendulum model. For the SimpleFOC reaction wheel pendulum, the state vector is $x = [\theta, \dot{\theta}, \phi]^T$ with input $u = \tau_m$ (motor torque). Wright [33] derived the linearized model via Lagrangian mechanics and identified parameters through system identification (oscillation period $\rightarrow$ inertia, logarithmic decrement $\rightarrow$ damping, torque-current characterization $\rightarrow K_t$). The resulting state-space matrices are:

$$A_{rw} = \begin{bmatrix} 0 & 1 & 0 \\ 30.86 & -0.58 & 0 \\ -30.86 & 0.58 & 0 \end{bmatrix}, \quad B_{rw} = \begin{bmatrix} 0 \\ -127.9 \\ 5652.8 \end{bmatrix}. \quad (5)$$

The structure mirrors the Furuta model: the A matrix encodes the gravitational instability (positive entry $a_{21} = 30.86$) and the B matrix encodes the motor torque coupling, but with three states instead of four. This illustrates how the same *Design-OS* workflow produces analogous but distinct state-space models for different platforms—both traceable to their respective parameters and requirements.

Control design (software domain, REQ-11). A state-feedback controller $u = K(x_d - x)$ is designed via pole placement. The dominant closed-loop poles are placed at $p_{1,2} = -\sigma \pm j\omega_d$ where $\sigma = \zeta\omega_n = 2.8$ and $\omega_d = \omega_n \sqrt{1 - \zeta^2} = 2.86$ rad/s, satisfying REQ-01 and REQ-02. The gain vector K is computed analytically from the desired characteristic polynomial, ensuring traceability from

TABLE 2: Simulation verification of performance requirements.

REQ	Specification	Simulation	Result
01	$\zeta = 0.7$	$\zeta = 0.700$	Pass
02	$\omega_n = 4$ rad/s	$\omega_n = 4.000$ rad/s	Pass
03	$ \alpha < 15^\circ$	$ \alpha _{\max} = 2.9^\circ$	Pass
04	$ V_m < 10$ V	$ V_m _{\max} = 0.5$ V	Pass

TABLE 3: Cross-domain traceability: requirement $\rightarrow$ domain flow $\rightarrow$ verification.

REQ	Domain flow	Verification
01, 02	Perf $\rightarrow$ SW (gain K) $\rightarrow$ Elec (V_m) $\rightarrow$ Mech (τ)	Sim. step resp.
03	Perf $\rightarrow$ Mech (pendulum α)	Sim. deflection
04	Perf $\rightarrow$ Elec (amplifier limit)	Sim. V_m
05	Mech + Elec $\rightarrow A, B$ matrices	Derivation
07	Elec (R_m, k_m, k_t, K_g) $\rightarrow A, B$	Review
11	SW (pole placement) $\rightarrow K$	Design rationale

performance specifications through the gain computation to the closed-loop response (REQ-11).

Simulation verification (REQ-01–04). A Python simulation was generated by the *Design-OS* /generate-simulation command directly from the specification artifacts. The script builds the state-space model from documented parameters, computes gain K via pole placement, and simulates the closed-loop step response. Table 2 summarizes the verification results: all four performance requirements pass. The simulation code, plots, and verification report are included in the repository alongside the specification artifacts, completing the traceability chain from requirements through design parameters to numerical verification.

4.4 Cross-Domain Traceability and Platform Comparison

Cross-domain requirement flow. A key insight is that performance requirements flow through all three domains. For example, REQ-01 ($\zeta = 0.7$) determines the closed-loop pole locations (software: pole placement $\rightarrow$ gain K), which determine the voltage command V_m (electrical: motor equation), which produces torque τ (mechanical: arm and pendulum dynamics). Table 3 summarizes this cross-domain traceability.

Platform comparison (feasibility gate output). As produced by the feasibility gate in Stage 3, Table 4 compares two implementations satisfying the same requirements: (1) the SimpleFOC-based reaction wheel pendulum (open-source, BLDC motor, Arduino) [31], and (2) the Quanser SRV02 with Furuta pendulum

TABLE 4: Platform comparison: same requirements, different implementations.

Domain	SimpleFOC	Quanser SRV02
Mechanical	3D-printed, sys. ID needed [33]	Aluminum arm + pendulum, known params
Electrical	BLDC + FOC driver, Arduino	DC servo, gearbox, commercial DAQ
Software	C++ + SimpleFOC lib.	MATLAB/Simulink, QUARC
Pendulum	Reaction wheel (2-DOF)	Furuta (2-DOF)
Cost	~\$100–200	~\$5k+

(commercial, proprietary, DC servo). Both satisfy the performance requirements (REQ-01–04), but through different mechanical configurations (reaction wheel vs. arm-driven Furuta, both 2-DOF), different electrical subsystems (BLDC with field-oriented control vs. DC servo), and different software stacks (Arduino/C++ vs. MATLAB/Simulink). The domain-specific requirements (REQ-05–12) make these differences explicit and traceable. Wright [33] validated a SimpleFOC-class reaction wheel pendulum with full system identification and state-feedback control, demonstrating that the open-source platform can achieve model-based control design comparable to commercial platforms. The feasibility gate found that the SimpleFOC platform costs ~50× less and removes licensing constraints, while requiring system identification for parameter characterization.

4.5 AI Model Comparison

To assess whether the *Design-OS* workflow produces consistent results across AI models, we ran the same pipeline (from plan-project through generate-simulation) with two proprietary models: Claude Opus 4.6 (Anthropic) and Gemini 3.1 Pro (Google). This comparison evaluates which stages each model handles well, where outputs diverge, and whether the structured commands are sufficient to guide consistent results across models. Table 5 rates each model’s output quality per *Design-OS* stage; Table 6 summarizes the type and frequency of human corrections required.

Early indicators suggest that the structured command format and explicit user checkpoints reduce model-dependent variation.

5 DISCUSSION

What worked. The *Design-OS* workflow proved effective for structuring the progression from context and literature through to requirements and implementation plan. Several aspects stood out. First, the specification served as the *single source of truth* and reduced ad hoc decisions during implementation: once requirements were formalized with IDs, “shall” statements, and verification methods, the controller design became a matter of

satisfying those requirements rather than iterating by trial and error. Second, the explicit literature stage prevented premature commitment to a solution. The conceptual design was informed by a landscape of prior work rather than by the first approach that came to mind. Third, the phased structure made the design process *auditable*: a reviewer (or instructor) can inspect the chain from mission to requirements to parameters to verification. Fourth, the literature-driven domain decomposition prevented the AI from fabricating system structure: each domain boundary was grounded in published work. Fifth, the validation gates surfaced cost and feasibility constraints early—particularly the Quanser vs. SimpleFOC comparison—preventing commitment to infeasible approaches. Sixth, user checkpoints ensured that the designer retained authority over decisions where literature was insufficient, supporting genuine human–AI teaming.

Comparison with typical practice. In typical control design coursework and many research projects, the path from performance specifications to controller gains is an in-paper or in-lab exercise: students receive specifications (e.g., damping ratio, settling time), compute gains, and report results. The mapping from intent to parameters exists but is rarely documented as a traceable workflow. *Design-OS* adds a thin but explicit layer of structure—concept definition, literature survey, conceptual design, requirements definition, design definition—that makes this mapping inspectable and reproducible. Compared to full MBSE or VDI 2206 processes, *Design-OS* is lighter weight and does not require specialized modeling languages (e.g., SysML, Modelica) or dedicated MBSE platforms—it operates on structured Markdown artifacts with AI models (e.g., Claude Opus 4.6) preferably inside an IDE (e.g., VS Code). Compared to ad hoc approaches, it provides structure to support traceability and reproducibility.

Limitations. Several limitations should be noted. First, maintaining the specification introduces overhead: writing and updating requirements, traceability tables, and implementation plans takes time that would otherwise be spent on implementation. For very small projects, this overhead may not be justified. Second, the workflow requires discipline in respecting stage order; in practice, designers may be tempted to skip ahead to implementation before finalizing the specification. Third, end-to-end traceability from intent to parameters remains challenging and tool-dependent—the workflow structures for it conceptually, but the degree to which it can be made fully explicit depends on the tools and environment. Fourth, the design case presented here is a well-understood benchmark. Applying *Design-OS* to novel or poorly understood systems where requirements are harder to specify upfront remains to be validated. Fifth, the validation gates as currently implemented are lightweight checklists rather than formal analysis; for safety-critical systems, more rigorous gate criteria would be needed. Sixth, the user checkpoints rely on the designer being available and engaged; in a fully asynchronous

TABLE 5: AI model output quality by *Design-OS* stage. Source data: log/report_claude.md, log/report_gemini.md.

	Claude Opus 4.6	Gemini 3.1 Pro
Overview	36 artifacts, 29 checkpoints, 24 corrections. Python (model choice). Platforms: Quanser SRV02 + SimpleFOC reaction wheel.	~25 artifacts, 28 checkpoints, 14 corrections. MATLAB (tech-stack). Platforms: Quanser QUBE + ESP32.
Stage 1: Concept Def.	Complete — 0 errors. Mission, roadmap, tech stack correct; open tech stack appropriately deferred.	Complete — 0 errors. Mission, roadmap, tech stack correct; MATLAB explicitly committed.
Stage 2: Lit. Survey	Good — 21 ref. errors caught by /verify-references; 8-theme coverage; pipeline step self-initiated.	Fair — 55 sources; 2 citation mismatches found by post-hoc /verify-references (2026-03-07); step not self-initiated.
Stage 3: Concept. Design	Good — 1 DOF factual error (1-DOF stated, 2-DOF correct); platform selection and functions correct.	Good — 1 domain error (Control/Math domain missing from decomposition); platforms, functions correct.
Stage 4: Req. Def.	Good — 12 requirements, 4 formal standards (/discover-standards); 2 minor corrections.	Poor — 7 requirements; /validate-feasibility gate skipped; /discover-standards not run; 3 specification errors: scope deviation, phantom REQ-HWW IDs, HIL in REQ-IMP-002.
Stage 5: Design Def.	Complete — both platforms ALL PASS; specification-driven order maintained; 1 numerical error (open-loop poles).	Fair — both platforms pass; execution order corrected; tool mismatch between MATLAB artifacts and Python report.

TABLE 6: Human corrections needed per model.

Correction	Claude Opus 4.6	Gemini 3.1 Pro
Factual errors	20	1
Missing reqs.	0	1
Halluc. refs.	3	2
Struct. deviat.	1	9
Domain errors	0	1
<i>Total</i>	24	14

workflow, the interruptions may disrupt flow.

Implications for education. For engineering design education, the workflow makes the design process *visible* and provides a structure within which students can be assessed not only on their final controller performance but on the quality of their requirements, the traceability of their design decisions, and the completeness of their documentation. The phased structure also provides natural checkpoints for instructor feedback. Whether this structure has the potential to ease learning is not evident from this paper and would require evaluation in a classroom or controlled setting. We plan such studies as future work. The blank template shared alongside this paper can be adapted to mechatronics and control systems courses; the inverted pendulum design case includes both a low-cost SimpleFOC reaction wheel platform and a commercial Quanser Furuta platform, giving instructors flexibility in hardware selection.

Implications for AI-assisted design. *Design-OS*’s phased artifacts are well-suited to AI-assisted workflows. Each stage produces structured documents (mission, literature notes, requirements, implementation plans) that can be generated or reviewed by large language models. The specification serves as a shared contract between human designers and AI agents, aligning with the philosophy of GitHub Spec Kit and Microsoft specification-driven development. Multi-agent frameworks such as MetaGPT [19] could be adapted to execute *Design-OS* stages, with agents assigned to literature search, conceptual design, specification elaboration, and implementation planning.

Future work. Future work includes applying *Design-OS* to additional design cases beyond the inverted pendulum family, extending the AI model comparison to open-weight models (e.g., Kimi K2.5), tightening integration with AI agents (e.g., automated literature summarization, specification-to-task decomposition, and requirement verification), and exploring tooling that better supports intent-to-parameter traceability. We also plan to evaluate the workflow in a classroom setting with student teams.

6 CONCLUSION

We presented *Design-OS*, a lightweight, specification-driven, phased workflow for engineering system design (concept definition → literature survey → conceptual design → requirements definition → design definition), enhanced with literature-driven domain decomposition, validation gates between stages, and user checkpoints for human–AI teaming. We positioned it rel-

ative to specification-driven and requirements-driven design—including industry practice such as GitHub Spec Kit and Microsoft specification-driven development—systematic design frameworks, and AI-assisted design. We demonstrated the workflow on an inverted pendulum control design project using two platforms—an open-source SimpleFOC reaction wheel pendulum and a commercial Quanser Furuta pendulum—showing how the same specification-driven workflow and requirements can be satisfied by fundamentally different implementations, how domain decomposition was driven by literature references, how validation gates surfaced cost and feasibility constraints, and how user checkpoints ensured designer authority over decisions where literature was insufficient. *Design-OS* extends the specification-driven multi-agent paradigm to physical engineering system design. We share a blank *Design-OS* template and the full design-case artifacts in a public GitHub repository [34] to demonstrate generality and support replicability testing. Future work will apply the workflow to further design cases, deepen integration with AI-assisted tools, evaluate in classroom settings, and conduct sandbox replicability testing across multiple AI models and independent users.

REFERENCES

- [1] Lee, M., Law, E., and Hoffman, R. R., 2024. “When and how to use AI in the design process?”. *International Journal of Human-Computer Interaction*, **41**(2), pp. 1569–1584.
- [2] Massoudi, S., and Fuge, M., 2025. “Agentic large language models for conceptual systems engineering and design”. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Vol. 89237, American Society of Mechanical Engineers, p. V03BT03A045.
- [3] Walden, D. D., Shortell, T. M., Roedler, G. J., Delicado, B. A., Mornas, O., Ip, Y.-S., and David, E., 2023. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th ed. Wiley, Hoboken, NJ.
- [4] Pahl, G., Beitz, W., Feldhusen, J., and Grote, K.-H., 2007. *Engineering design: a systematic approach*, 3rd ed. Springer.
- [5] Hamza, M. F., Yap, H. J., Choudhury, I. A., Isa, A. I., Zimit, A. Y., and Kumbasar, T., 2019. “Current development on using rotary inverted pendulum as a benchmark for testing linear and nonlinear control algorithms”. *Mechanical Systems and Signal Processing*, **116**, pp. 347–369.
- [6] GitHub, 2025. GitHub spec kit. <https://github.com/github/spec-kit>. Accessed: 2026-02-01.
- [7] Microsoft, 2025. Specification-driven development: Spec Kit. <https://developer.microsoft.com/blog/spec-driven-development-spec-kit>. Accessed: 2026-02-01.
- [8] Gräßler, I., and Hentze, J., 2020. “The new V-model of VDI 2206 and its validation”. *at-Automatisierungstechnik*, **68**(5), pp. 312–324.
- [9] Cooper, R. G., 1990. “Stage-gate systems: a new tool for managing new products”. *Business Horizons*, **33**(3), pp. 44–54.
- [10] Nuseibeh, B., and Easterbrook, S., 2000. “Requirements engineering: a roadmap”. In Proceedings of the Conference on the Future of Software Engineering, ACM, pp. 35–46.
- [11] Zave, P., and Jackson, M., 1997. “Four dark corners of requirements engineering”. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, **6**(1), pp. 1–30.
- [12] Gotel, O., and Finkelstein, A., 1994. “An analysis of the requirements traceability problem”. In Proceedings of IEEE International Conference on Requirements Engineering, IEEE, pp. 94–101.
- [13] Ramesh, B., and Jarke, M., 2001. “Toward reference models for requirements traceability”. *IEEE Transactions on Software Engineering*, **27**(1), pp. 58–93.
- [14] IEEE, 1998. “IEEE Recommended Practice for Software Requirements Specifications”. *IEEE Std 830-1998*, pp. 1–40.
- [15] ISO/IEC/IEEE, 2018. “ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering”. *ISO/IEC/IEEE 29148:2018(E)*, pp. 1–104.
- [16] Mavin, A., Wilkinson, P., Harwood, A., and Novak, M., 2009. “Easy approach to requirements syntax (EARS)”. In 2009 17th IEEE international requirements engineering conference, IEEE, pp. 317–322.
- [17] Piskala, D. B., 2026. Spec-driven development: from code to contract in the age of AI coding assistants. arXiv:2602.00180.
- [18] Gero, J. S., 1990. “Design prototypes: a knowledge representation schema for design”. *AI Magazine*, **11**(4), pp. 26–36.
- [19] Hong, S., Zheng, X., Chen, Y., Cheng, J., Zhang, C., Wang, Z., Xu, Q., et al., 2024. “MetaGPT: meta programming for a multi-agent collaborative framework”. In International Conference on Learning Representations (ICLR).
- [20] Qian, C., et al., 2024. “ChatDev: communicative agents for software development”. In Proceedings of the ACL, pp. 15174–15186.
- [21] Zadenoori, M. A., Dąbrowski, J., Alhoshan, W., Zhao, L., and Ferrari, A., 2025. Large language models for requirements engineering: a systematic review. arXiv:2509.11446.
- [22] Ma, K., Grandi, D., McComb, C., and Goucher-Lambert, K., 2023. “Conceptual design generation using large language models”. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Vol. 87349, American Society of Mechanical Engineers, p. V006T06A021.
- [23] Zhu, Q., and Luo, J., 2022. “Generative pre-trained transformers (GPT) for design concept generation: an exploration”. *Proceedings of the design society*, **2**, pp. 1825–1834.
- [24] Awtar, S., King, N., Allen, T., Bang, I., Hagan, M. D., Skidmore, D., and Craig, K., 2002. “Inverted pendulum systems: rotary and arm-driven—a mechatronic system design case study”. In *Mechatronics*, Vol. 12, pp. 357–370.
- [25] Boubaker, O., 2013. “The inverted pendulum benchmark in nonlinear control theory: a survey”. *International Journal of Advanced Robotic Systems*, **10**(5).
- [26] Dormido Bencomo, S., 2004. “Control learning: present and future”. *Annual Reviews in Control*, **28**(1), pp. 115–136.
- [27] Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S., 2007. “A design science research methodology for information systems research”. *Journal of Management Information Systems*, **24**(3), pp. 45–77.
- [28] Furuta, K., Yamakita, M., and Kobayashi, S., 1992. “Swing-up control of inverted pendulum using pseudo-state feedback”. *Proceedings of the Institution of Mechanical Engineers, Part I*, **206**(6), pp. 263–269.
- [29] Quanser, 2026. Research papers. Quanser Community.
- [30] Feedback Instruments, 2025. Digital pendulum system 33-005-PCI. Feedback Instruments (distributed by LD Didactic/Leybold).
- [31] SimpleFOC Community, 2023. Reaction wheel inverted pendulum: Arduino SimpleFOC.
- [32] Skuric, A., Bank, H. S., Unger, R., Williams, O., and González-Reyes, D., 2022. “SimpleFOC: a field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors”. *Journal of Open Source Software*, **7**(74), p. 4232.
- [33] Wright, Z., 2023. “Design and implementation of a prototype pendulum platform for aerospace controls education”. M.S. thesis, California Polytechnic State University, San Luis Obispo, June.
- [34] Bank, H. S., Herber, D. R., and Bradley, T. H., 2026. Design-OS: Specification-driven framework for engineering system design. <https://github.com/bankh/design-os>. Accessed: 2026-03-12.
- [35] Cazzolato, B. S., and Prime, Z., 2011. “On the dynamics of the

A DESIGN-OS COMMANDS

Table 7 lists the *Design-OS* commands, organized by workflow stage. Each command is a structured Markdown prompt file that defines the stage’s process steps, expected inputs and outputs, and validation checks. Commands are executed by invoking them as slash commands (e.g., `/plan-project`) in an IDE with appropriate AI model support (e.g., VS Code). They are model-agnostic by design. The full prompt files are available in the public repository [34] along with detailed explanations of the runtime requirements and individual commands.

The remainder of this appendix provides detailed definitions of each command. Each command file follows a common template: (1) a preamble stating the command’s role in the workflow, (2) important guidelines that constrain the agent’s behavior, (3) a workflow validation block that checks prerequisites before execution begins, and (4) a numbered process with 5–10 steps, each producing a named artifact.

A.1 Stage 1: Concept Definition

/plan-project. Establishes the foundational design context through interactive dialogue with the designer. The command walks through three areas: (1) the design *mission*—problem statement, stakeholders, success criteria, and constraints; (2) a *roadmap* defining the project phases, milestones, and deliverables; (3) the *tech stack*—tools, languages, simulation environments, and hardware platforms. The agent asks one question at a time, confirms each answer, and writes the results to three files (`mission.md`, `roadmap.md`, `tech-stack.md`) in the project folder. If existing project documents are found, the command reads them first and proposes updates rather than starting from scratch. The output forms the input context for all subsequent stages. Interactive checkpoints include: *What system or design problem are we addressing?*, *Who are the stakeholders or users?*, *What constraints or success criteria matter?*, *What are the main design phases or milestones?*, *What comes first and what follows?*, *What tools does the project use?*, and *What formal methods or frameworks are you using?*

A.2 Stage 2: Literature Survey

/literature-search. Runs an initial landscape literature search aligned with the project mission and roadmap. The command reads the Stage 1 artifacts, then produces three outputs: (1) *deep-research prompts*—structured queries designed to be executed by AI research assistants (e.g., deep research tools) that request a single comprehensive report covering all relevant aspects of the design problem; (2) *scholar search queries*—keyword and Boolean queries formatted for academic search engines (Google Scholar, IEEE Xplore, etc.); (3) a *preliminary literature note* synthesizing what is known from the agent’s training data, explicitly marking claims that require verification. The scope is deliberately preliminary: the command surfaces the landscape and identifies gaps rather than producing a final literature review. The designer is asked: *What topics or domains should the literature search cover?*

/focused-literature (after Stage 3). Runs a targeted literature search driven by the conceptual design output. Unlike the initial landscape search, this command focuses on specific gaps, domain boundaries, or technical questions identified during conceptual design and domain decomposition. It requires the conceptual design artifacts as input, generates focused deep-research prompts and scholar queries scoped to the identified gaps, and produces a focused literature report. The command also supports an optional standards search when the conceptual design identifies regulatory or normative requirements. The designer is asked to *confirm or adjust the focus areas* before the search proceeds.

/verify-references (after focused literature). Verifies all references cited in the preliminary and focused literature notes by web search. The

command reads the literature artifacts, extracts every cited reference (author, title, year, venue), and checks each against web sources (publisher pages, Google Scholar, DOI resolvers). It produces an append-only *corrections log* documenting: (1) hallucinated references (no matching publication found), (2) wrong-author attributions, (3) incorrect years or venues, and (4) minor errors (initials, title wording). The corrections log ensures that fabricated citations do not propagate into the specification. This command is fully automated and does not require user checkpoints.

A.3 Stage 3: Conceptual Design

/conceptual-design. Transforms the literature and project context into a first conceptual design. The command proceeds through: (1) formulating *design objectives* (DOs); (2) identifying candidate *methodologies* and approaches grounded in the literature; (3) defining *functions and needs* the system must satisfy; (4) generating *concepts* (candidate solutions or architectures); (5) proposing a *high-level structure* (e.g., paper outline, system architecture, or module breakdown). The command explicitly requires that design objectives and methodologies are fixed before concepts are generated, enforcing the principle that the problem framing precedes solution generation. Interactive checkpoints include: *What are the primary design objectives?*, *What formal methodologies will we use?*, *What are the main functions and stakeholder needs?*, *What solution concepts or platforms will we target?*, and a *literature sufficiency check*.

/domain-decomposition. Identifies the constituent domains of the system—typically mechanical, electrical, and software for mechatronic systems—with every proposed domain boundary traceable to at least one literature reference from Stage 2. The command reads the literature and conceptual design artifacts, proposes domains with citations, identifies *inter-domain interfaces* (e.g., the torque equation coupling electrical voltage to mechanical torque), and performs a *gap assessment* that flags domain boundaries or interfaces where literature is insufficient. When gaps are found, the command surfaces a user checkpoint rather than fabricating justifications. Interactive checkpoints include: *Are these domains correct?*, *Should any be added, merged, or removed?*, *What are the inter-domain interfaces?*, and a *gap assessment* asking whether to flag gaps or provide information. The decomposition output feeds directly into Stage 4 (Requirements Definition), where requirements are organized by domain.

A.4 Gate

/validate-feasibility. Runs a feasibility gate check between major stages. The command can be invoked at any gate and assesses four dimensions: (1) *technical feasibility*—whether the proposed approach is realizable with the chosen architecture, components, and methods; (2) *cost assessment*—comparing platform and component costs (e.g., commercial vs. open-source hardware); (3) *licensing and IP*—evaluating software licensing constraints (proprietary vs. open-source), patent considerations, and data access; (4) *availability and procurement*—checking whether required components, platforms, or tools are accessible. The gate produces a feasibility report with candidate constraints that may become requirements in Stage 4. Interactive checkpoints include: *Are any technical concerns blockers?*, *Cost—estimates and budget constraints?*, and a *go/no-go decision* on whether to proceed, iterate, or adjust scope.

A.5 Stage 4: Requirements Definition

/spec. Structures the specification by defining section-level requirements and the spec layout. This command must be run in plan mode (i.e., the agent plans the spec structure before writing it). It reads the conceptual design, domain decomposition, and literature artifacts, then produces: (1) a *shape* document defining the spec sections, their scope, and the requirement categories; (2) a *standards* document listing relevant standards and conventions discovered during the process; (3) a *references* document linking spec sections to their source literature. The spec folder structure is generated with placeholder files for requirements, interfaces, validation criteria, and traceability. Interactive checkpoints include: *What are we designing?*, *Are visuals needed?*, *What reference implementations*

TABLE 7: *Design-OS* commands by stage.

Stage	Command	Purpose
1: Concept Definition	<code>/plan-project</code>	Establish mission, roadmap, and tech stack through interactive dialogue
2: Literature Survey	<code>/literature-search</code>	Run landscape search and produce lit note, deep-research prompts, and scholar queries
	<code>/focused-literature</code>	Targeted search driven by conceptual design gaps
	<code>/verify-references</code>	Web-verify cited references and log corrections
3: Conceptual Design	<code>/conceptual-design</code>	Develop design objectives (DOs), methodologies, and high-level structure from literature
	<code>/domain-decomposition</code>	Identify system domains and interfaces from literature references
Gate	<code>/validate-feasibility</code>	Gate check: cost, licensing, component availability, technical feasibility
4: Requirements Def.	<code>/spec</code>	Structure section-level requirements and specification layout (plan mode)
	<code>/discover-standards</code>	Extract recurring design patterns into documented standards
	<code>/index-standards</code>	Rebuild standards index for quick matching by <code>/inject-standards</code>
	<code>/inject-standards</code>	Apply relevant standards to current work context
5: Design Definition	<code>/plan-implementation</code>	Derive task breakdown and dependency ordering from specification
	<code>/generate-simulation</code>	Generate executable verification code from specification and model

apply?, and a *consistency check* against existing project artifacts. The spec output serves as the scaffold for the elaborate-spec step (manual or command-driven), where detailed requirements with IDs, “shall” statements, priority, source, and verification method are filled in.

/discover-standards. Extracts recurring design patterns from the project into concise, documented standards stored in a dedicated `standards/` directory. The command analyzes the current project artifacts (code, specs, documentation), identifies patterns that should be codified (e.g., requirements format, naming conventions, system structure templates), and for each proposed standard asks the designer *why* the pattern matters before writing it. Standards are indexed in a YAML file for discoverability. The designer is asked: *Which areas should be documented as standards?* This command can be run at any point during the workflow but is most useful during or after Stage 4 (Requirements Definition) when patterns have emerged.

/index-standards. Rebuilds and maintains the standards index file (`standards/index.yml`). The index maps each standard to a brief description, enabling `/inject-standards` to suggest relevant standards without reading all files. The command scans all Markdown files in the `standards/` directory, compares them against the existing index, and for each new file proposes a one-sentence description for the designer to confirm. Deleted files are removed from the index. The command is also run automatically as the final step of `/discover-standards`. Interactive checkpoints include confirming or editing the proposed description for each new standard.

/inject-standards. Applies relevant standards from the `standards/` directory to the current work context. The command reads the standards index, analyzes the current task or artifact being worked on, matches applicable standards,

and presents them to the designer with a recommendation. Standards can be injected automatically (e.g., when writing a new spec section, the requirements format standard is surfaced) or explicitly by the designer. This command ensures consistency across artifacts and stages without requiring the designer to manually consult the standards directory.

A.6 Stage 5: Design Definition

/plan-implementation. Turns an elaborated spec into a concrete implementation plan and task list. The command reads the spec folder (requirements, interfaces, validation criteria, traceability), detects the project context (e.g., control system, software module, mechatronic subsystem), and produces: (1) an *implementation plan* with ordered tasks, each tied to specific requirements and deliverables; (2) a *dependency graph* showing which tasks must precede others; (3) a *task list* (e.g., a checklist or todo list) for execution tracking. The designer confirms *artifact structure, execution order, and deliverables*. The command ensures that every requirement in the spec has at least one corresponding implementation task.

/generate-simulation. Generates executable simulation code directly from the elaborated spec artifacts. The command reads the requirements document (specifically performance requirements with numerical targets), the system model (state-space matrices, transfer functions, or equations of motion), and the traceability table, then produces simulation code that: (1) builds the system model from documented parameters, (2) implements the control law or design under test, (3) runs verification scenarios (e.g., step response, disturbance rejection), and (4) checks each performance requirement against simulation results, reporting pass/fail with numerical values. The command chooses the simulation language based on the tech stack (e.g., Python or MATLAB) and ensures that the generated code is self-contained, reproducible, and directly traceable to the spec. This command is fully automated and does not require user checkpoints.